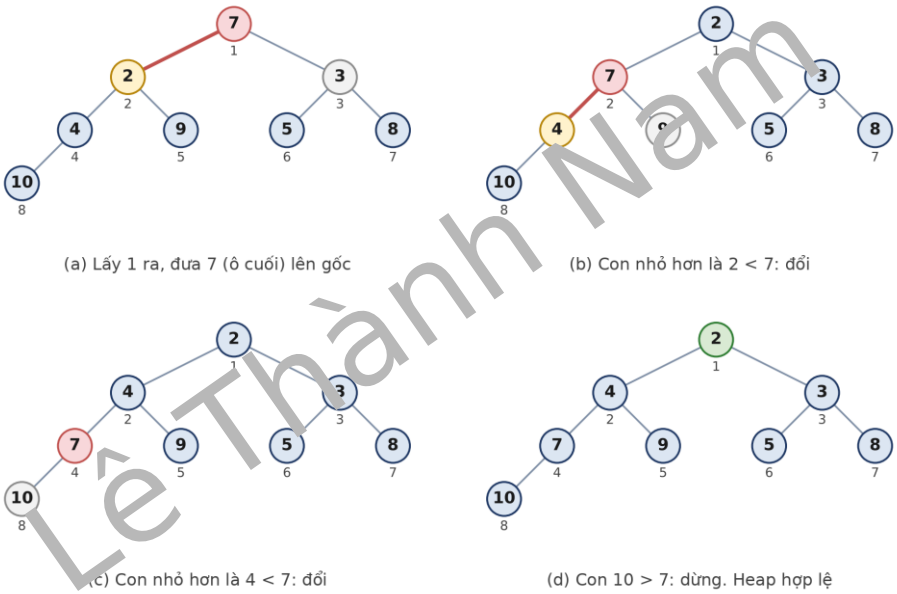
Tổng hợp và biên soạn: Lê Thành Nam

TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

CẤU TRÚC DỮ LIỆU DẠNG CÂY

Từ cơ bản đến chuyên sâu

TẬP 3 · HƯỚNG DẪN GIẢI



Lê Thành Nam

Chương 1. CÁC CẤU TRÚC CÂY CƠ BẢN
Chương 2. CÂY PHÂN ĐOẠN NÂNG CAO
Chương 3. CÂY CÂN BẰNG
Chương 4. CÂY TRÊN XÂU
Chương 5. CÁC CẤU TRÚC CÂY ĐẶC BIỆT

LỜI NÓI ĐẦU

Tập 3 trình bày lời giải cho 66 bài tập của Tập 2. Mỗi lời giải không chỉ đưa ra chương trình đạt 100 điểm, mà đi lại đúng con đường một học sinh giỏi suy nghĩ trong phòng thi: bắt đầu từ cách ngây thơ có sơ đồ khối, đếm số phép tính để thấy bế tắc, rồi phá bế tắc từng bước bằng một quan sát hay một cấu trúc dữ liệu phù hợp.

Mọi chương trình trong sách đều đã được kiểm tra hai lần: đối chiếu với lời giải vét cạn trên hàng trăm bộ dữ liệu nhỏ ngẫu nhiên, và chấm trên bộ test chính thức với giới hạn 1 giây, 256 MB. Các con số trong bảng điểm cuối mỗi bài là điểm thực tế mà từng chương trình nhận được trên bộ test đó, không phải ước lượng.

Lê Thành Nam

Lê Thành Nam

CẤU TRÚC MỘT LỜI GIẢI

Mục	Nội dung
1. Hiểu bài toán	Diễn đạt lại đề bằng ngôn ngữ của cấu trúc dữ liệu: cần những thao tác nào, trên dữ liệu nào, bao nhiêu lần.
2. Cách ngây thơ	Sơ đồ khối, chương trình và độ phức tạp của cách làm đầu tiên ai cũng nghĩ ra.
3. Bế tắc	Vì sao cách hiện tại không vượt qua được subtask lớn; đôi khi kèm một lời giải trung gian hoặc một lời giải sai "trông như đúng".
4. Phá bế tắc	Từng bước: quan sát, cấu trúc dữ liệu, hình vẽ chạy tay, lập luận vì sao đúng.
5. Cách tối ưu	Sơ đồ khối hoặc code ý chính, chương trình hoàn chỉnh và độ phức tạp.
6. Bẫy thường gặp	Những lỗi làm mất điểm hay gặp nhất.
7. Bảng điểm	Điểm thực tế của từng cách làm trên bộ test. Dòng tô xanh là lời giải đạt trọn điểm.

Lời khuyên: Đừng đọc lời giải như đọc truyện. Sau mục 3, hãy gấp sách và tự tìm cách phá bế tắc trong ít nhất mười lăm phút. Sau mục 5, hãy tự viết lại chương trình mà không nhìn sách, rồi tự sinh test để đối chiếu với cách ngây thơ của chính em.

Lê Thành Nam

MỤC LỤC

LỜI NÓI ĐẦU **2**

CẤU TRÚC MỘT LỜI GIẢI **3**

CHƯƠNG 1. CÁC CẤU TRÚC CÂY CƠ BẢN **6**

Bài 1.1. TRUNG VỊ 7
Bài 1.2. TỔNG NHỎ 14
Bài 1.3. CÔNG VIỆC 20
Bài 2.1. KHO HÀNG 28
Bài 2.2. XẾP HÀNG 33
Bài 2.3. ĐOẠN CON 38
Bài 3.1. SƠN TƯỜNG 43
Bài 3.2. BẢN ĐỒ 48
Bài 3.3. VÒNG TRÒN 53
Bài 4.1. MIN VÀ ĐẾM 59
Bài 4.2. DÃY NGOẶC 64
Bài 4.3. ĐOẠN LỚN NHẤT 70
Bài 5.1. GÁN ĐOẠN 76
Bài 5.2. BẬT TẮT ĐÈN 81
Bài 5.3. CẤP SỐ CỘNG 86
Bài 6.1. THÀNH PHỐ 91
Bài 6.2. CHÊNH LỆCH 96
Bài 6.3. ĐỒNG BỘ 101
Bài 7.1. BÃI ĐỖ XE 108
Bài 7.2. KHOẢNG CÁCH 112
Bài 7.3. ĐOẠN PHỦ 117

CHƯƠNG 2. CÂY PHÂN ĐOẠN NÂNG CAO **122**

Bài 8.1. XẾP CHỖ 123
Bài 8.2. SƠN TƯỜNG 128
Bài 8.3. KHÁCH SẠN 134
Bài 9.1. LỊCH SỬ 141
Bài 9.2. ĐẾM ĐOẠN 146
Bài 9.3. ĐẾM KHÁC 151
Bài 10.1. TỔNG DƯỚI 157
Bài 10.2. LƯỚI 162
Bài 10.3. ĐẾM CẬP NHẬT 167
Bài 11.1. CAO NHẤT 174
Bài 11.2. ẾCH NHẢY 179
Bài 11.3. ĐOẠN THẲNG 184
Bài 12.1. CĂN BẬC HAI 190
Bài 12.2. CHIA DƯ 195
Bài 12.3. TỔNG HỢP 201

CHƯƠNG 3. CÂY CÂN BẰNG **209**

Bài 13.1. HÀNG ĐỢI 210

Lê Thành Nam

Bài 13.2. ĐẢO NGƯỢC ... 216
Bài 13.3. CẮT DÁN ... 222
Bài 14.1. LÂN CẬN ... 228
Bài 14.2. ĐỔI ĐẦU ... 234
Bài 14.3. CHUỖI SỐ ... 240
Bài 15.1. HẠNG THI ... 246
Bài 15.2. SAU X ... 249
Bài 15.3. TRUNG VỊ TRƯỢT ... 254
CHƯƠNG 4. CÂY TRÊN XÂU ... 258
Bài 16.1. XOR LỚN NHẤT ... 259
Bài 16.2. XOR NHỎ HƠN K ... 263
Bài 16.3. XOR TRÊN ĐOẠN ... 268
Bài 17.1. TỪ CẤM ... 273
Bài 17.2. TRÁNH TỪ ... 278
Bài 17.3. MẪU ĐỘNG ... 283
Bài 18.1. KHÁC NHAU ... 289
Bài 18.2. CHUNG DÀI NHẤT ... 294
Bài 18.3. THỨ K ... 299
Bài 19.1. ĐẾM ĐỐI XỨNG ... 305
Bài 19.2. TRỌNG SỐ ... 310
Bài 19.3. CẮT ĐỐI XỨNG ... 315
CHƯƠNG 5. CÁC CẤU TRÚC CÂY ĐẶC BIỆT ... 320
Bài 20.1. ĐẾM KHOẢNG ... 321
Bài 20.2. TẦN SUẤT ... 325
Bài 20.3. HOÁN VỊ ... 330
Bài 21.1. NÉN ẢNH ... 337
Bài 21.2. TRẠM PHÁT ... 342
Bài 21.3. ĐẾM HÌNH CHỮ NHẬT ... 348
Bài 22.1. BẬT TẮT ... 353
Bài 22.2. LỊCH TRỐNG ... 358
Bài 22.3. ĐOẠN TỐI ... 364

Lê Thành Nam

CHƯƠNG 1. CÁC CẤU TRÚC CÂY CƠ BẢN

Ở chương này, phần lớn các bài có cùng một kịch bản: Cách ngây thơ làm mỗi thao tác trong O(n) bằng cách duyệt mảng, tổng cộng $O(n^2)$ hoặc $O(n \cdot q)$; cách tối ưu thay vòng duyệt đó bằng một cấu trúc cây để mỗi thao tác chỉ còn O(log n). Điều quan trọng nhất cần luyện là nhận ra **thao tác nào đang bị lặp lại** trong cách ngây thơ, vì đó chính là thao tác cần cấu trúc dữ liệu tăng tốc.

Lê Thành Nam

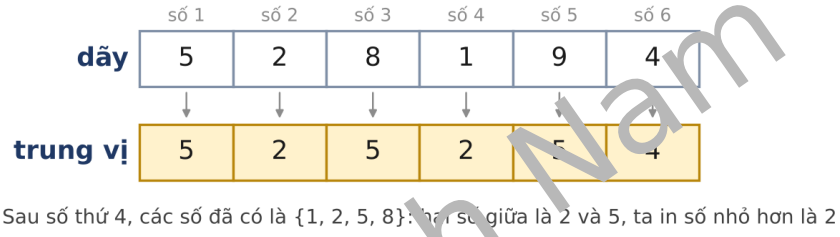
BÀI 1.1 TRUNG VỊ

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
TRUNGVI.INP	TRUNGVI.OUT	1 giây	256 MB	★ CƠ BẢN

Liên hệ: Bài 1

1. Hiểu bài toán

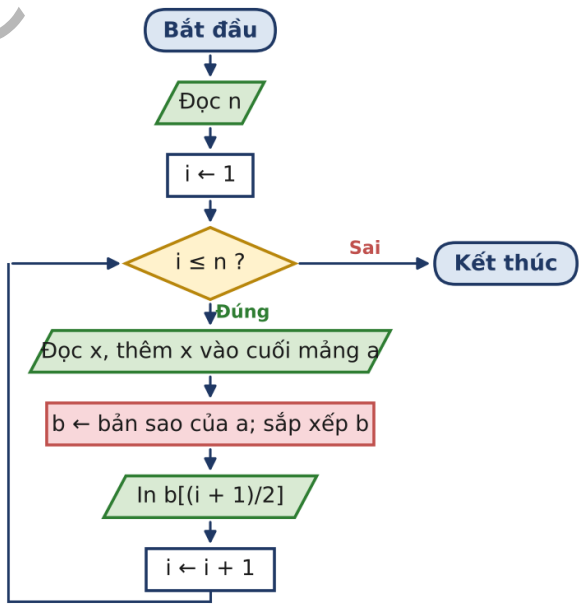
Ta có một tập số lớn dần: sau lần nhận thứ i, tập có i phần tử, và ta cần phần tử đứng thứ ⌈i/2⌉ theo thứ tự tăng dần. Như vậy có hai thao tác xen kẽ nhau n lần: **thêm một số** và **hỏi phần tử ở giữa**. Khó ở chỗ "phần tử ở giữa" thay đổi liên tục: số mới có thể rơi vào nửa nhỏ, đẩy trung vị lùi xuống, hoặc rơi vào nửa lớn, kéo trung vị tiến lên.



Hình 1.1.1. *Dãy của ví dụ 1 và trung vị sau mỗi lần nhận*

2. Cách ngây thơ: Sắp xếp lại sau mỗi lần nhận

Cách đơn giản nhất: cất các số vào mảng; sau mỗi lần nhận thêm số, sao chép mảng ra, sắp xếp bản sao và in phần tử thứ ⌈i/2⌉.



Hình 1.1.2. *Sơ đồ khối cách sắp xếp lại sau mỗi lần nhận*

Lê Thành Nam

Cách ngây thơ: Sắp xếp lại, O(n² log n)

```cpp
#include <cstdio>
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("TRUNGVI.INP", "r")) {
        freopen("TRUNGVI.INP", "r", stdin);
        freopen("TRUNGVI.OUT", "w", stdout);
    }
    int n;
    cin >> n;
    vector<long long> a;
    for (int i = 1; i <= n; i++) {
        long long x;
        cin >> x;
        a.push_back(x);
        vector<long long> b = a;
        sort(b.begin(), b.end());
        cout << b[(i + 1) / 2 - 1] << '\n';
    }
    return 0;
}
```

Độ phức tạp: Lần thứ i sắp xếp i số tốn khoảng $i \cdot \log_2 i$ phép so sánh. Tổng cộng $1 \cdot \log 1 + 2 \cdot \log 2 + \ldots + n \cdot \log n \approx (n^2/2) \cdot \log_2 n$. Với n = 1000: khoảng $5 \cdot 10^6$ phép tính, chạy tức thì. Với $n = 2 \cdot 10^4$: khoảng $2{,}9 \cdot 10^9$ phép tính, quá chậm.

3. Bế tắc

- Subtask 1 ($n \le 1000$): sắp xếp lại chạy tức thì: **30 điểm**.
- Subtask 2 và 3: quá giới hạn thời gian: **0 điểm**.

Cải tiến đầu tiên ai cũng nghĩ ra: đừng sắp xếp lại từ đầu, vì trước khi thêm số mới mảng **đã** được sắp xếp. Chỉ cần tìm chỗ đứng của số mới bằng tìm kiếm nhị phân (`upper_bound`) rồi chèn vào đó:

Cách trung gian: Chèn vào mảng đã sắp xếp, O(n²)

```cpp
b.insert(upper_bound(b.begin(), b.end(), x), x);
cout << b[(i + 1) / 2 - 1] << '\n';
```

Tìm chỗ chỉ tốn O(log i), nhưng lệnh `insert` phải dồn tất cả các phần tử phía sau lùi một ô, trung bình i/2 phép gán. Tổng cộng khoảng $n^2/4$ phép gán: với $n = 2 \cdot 10^4$ là 10^8 (máy dồn bộ nhớ rất nhanh nên vẫn kịp), với $n = 2 \cdot 10^5$ là 10^{10}, quá chậm. Cách này được **60 điểm**.

ĐIỂM BẾ TẮC

Cả hai cách đều giữ **toàn bộ** dãy ở trạng thái sắp xếp, trong khi mỗi lần ta chỉ hỏi đúng một phần tử ở giữa. Giữ thứ tự của mọi cặp phần tử là quá thừa, và cái giá của sự thừa đó là O(n) cho mỗi lần thêm.

Lê Thành Nam

4. Phá bế tắc: Hai heap giữ hai nửa

Bước 1 Trung vị là ranh giới giữa hai nửa

Hãy chia các số đã nhận thành hai nhóm: **nửa nhỏ** gồm ⌈i/2⌉ số nhỏ nhất, **nửa lớn** gồm các số còn lại. Trung vị chính là số **lớn nhất của nửa nhỏ**. Ta không cần biết thứ tự bên trong mỗi nửa; ta chỉ cần biết số lớn nhất của nửa nhỏ, và (để cân bằng khi cần) số nhỏ nhất của nửa lớn. Đó đúng là việc của heap!

Bước 2 Bất biến cần giữ

Dùng heap lớn nhất `lo` cho nửa nhỏ và heap nhỏ nhất `hi` cho nửa lớn. Ta luôn giữ hai điều kiện:

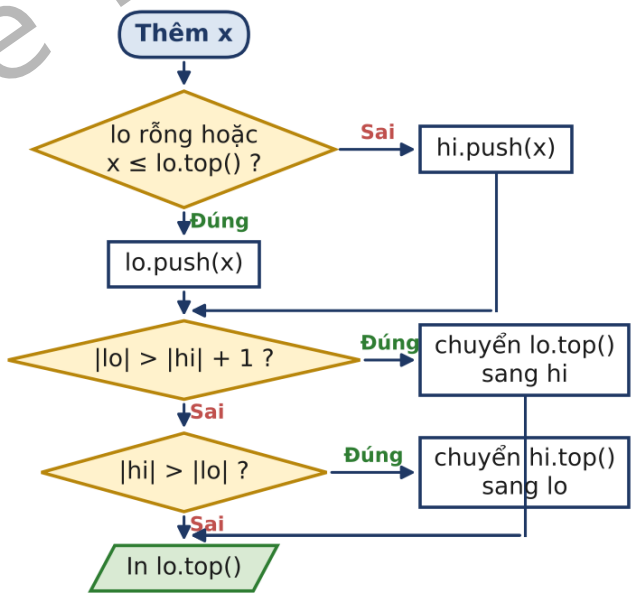
1. **Điều kiện thứ tự:** Mọi số trong `lo` không lớn hơn mọi số trong `hi`, tức là lo.top() ≤ hi.top().
2. **Điều kiện kích thước:** |lo| = |hi| hoặc |lo| = |hi| + 1.

Khi cả hai điều kiện đúng, `lo` chứa đúng ⌈i/2⌉ số nhỏ nhất, nên trung vị là `lo.top()`.

Bước 3 Thêm một số mà vẫn giữ bất biến

Khi nhận số x:

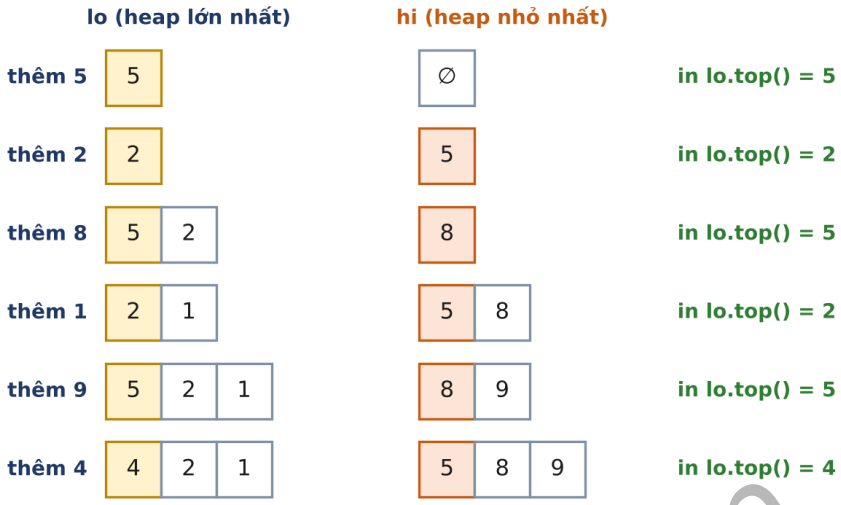
- Nếu `lo` rỗng hoặc x ≤ lo.top(), x thuộc nửa nhỏ: đưa vào `lo`. Ngược lại đưa vào `hi`. Điều kiện thứ tự vẫn đúng.
- Lúc này một heap có thể thừa đúng một phần tử. Nếu |lo| = |hi| + 2, chuyển lo.top() sang `hi`. Nếu |hi| = |lo| + 1, chuyển hi.top() sang `lo`. Vì phần tử được chuyển là phần tử "sát ranh giới" nhất, điều kiện thứ tự vẫn đúng.



Hình 1.1.3. *Sơ đồ khối thao tác thêm một số vào hai heap*

Lê Thành Nam

Bước 4 Chạy tay



Hình 1.1.4. *Nội dung hai heap sau mỗi lần thêm (ô màu là ảnh heap)*

Số	Việc làm	Kết quả
5	lo rỗng: vào lo	lo = {5}, hi = ∅; in 5
2	$2 \le 5$: vào lo; $\|lo\| = 2 > \|hi\| + 1$: chuyển 5 sang hi	lo = {2}, hi = {5}; in 2
8	$8 > 2$: vào hi; $\|hi\| = 2 > \|lo\| = 1$: chuyển 5 sang lo	lo = {5, 2}, hi = {8}; in 5
1	$1 \le 5$: vào lo; $\|lo\| = 3 > \|hi\| + 1$: chuyển 5 sang hi	lo = {2, 1}, hi = {5, 8}; in 2
9	$9 > 2$: vào hi; $\|hi\| = 3 > \|lo\|$: chuyển 5 sang lo	lo = {5, 2, 1}, hi = {8, 9}; in 5
4	$4 \le 5$: vào lo; $\|lo\| = 4 > \|hi\| + 1$: chuyển 5 sang hi	lo = {4, 2, 1}, hi = {5, 8, 9}; in 4

Các số in ra 5, 2, 5, 2, 5, 4 khớp với ví dụ.

> **VÌ SAO CHỈ CẦN CHUYỂN MỘT PHẦN TỬ?**
> Trước khi thêm, hiệu $|lo| - |hi|$ là 0 hoặc 1. Thêm một số làm hiệu thay đổi đúng 1 đơn vị, nên sau khi thêm hiệu thuộc $\{-1, 0, 1, 2\}$. Hiệu bằng 2 thì chuyển một phần tử từ lo sang hi (hiệu thành 0); hiệu bằng −1 thì chuyển một phần tử từ hi sang lo (hiệu thành 1). Không bao giờ phải chuyển hai phần tử.

Lê Thành Nam

5. Cách tối ưu

TRUNGVI.CPP: Hai heap, O(n log n)

```cpp
#include <cstdio>
#include <iostream>
#include <queue>
#include <vector>
#include <functional>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("TRUNGVI.INP", "r")) {
        freopen("TRUNGVI.INP", "r", stdin);
        freopen("TRUNGVI.OUT", "w", stdout);
    }
    int n;
    cin >> n;
    priority_queue<long long> lo;
    priority_queue<long long, vector<long long>, greater<long long> > hi;
    for (int i = 1; i <= n; i++) {
        long long x;
        cin >> x;
        if (lo.empty() || x <= lo.top()) lo.push(x);
        else hi.push(x);
        if (lo.size() > hi.size() + 1) {
            hi.push(lo.top());
            lo.pop();
        } else if (hi.size() > lo.size()) {
            lo.push(hi.top());
            hi.pop();
        }
        cout << lo.top() << '\n';
    }
    return 0;
}
```

Lê Thành Nam

Độ phức tạp: Mỗi số gây ra tối đa 2 lần push và 1 lần pop, mỗi lần O(log n). Tổng O(n log n): với $n = 2 \cdot 10^5$ khoảng 10^7 phép tính.

6. Bẫy thường gặp

- Lấy nhầm trung vị khi số phần tử chẵn: đề yêu cầu số **nhỏ hơn** trong hai số giữa, đó là lo.top(). Nếu giữ |hi| = |lo| + 1 thì phải in hi.top() và sẽ sai khi i chẵn.
- Gọi lo.top() khi `lo` rỗng ở lần thêm đầu tiên. Phải kiểm tra `lo.empty()` trước.
- Khai báo cả hai heap là heap lớn nhất: `hi` phải là heap nhỏ nhất (dùng `greater`).
- In bằng `endl` với $2 \cdot 10^5$ dòng làm chương trình chậm đáng kể; dùng `'\n'`.

7. Bảng điểm theo thuật toán

Thuật toán	Subtask 1 (30)	Subtask 2 (30)	Subtask 3 (40)	Tổng
Sắp xếp lại sau mỗi số, $O(n^2 \log n)$	30	0	0	30
Chèn vào mảng đã sắp xếp, $O(n^2)$	30	30	0	60
Hai heap, O(n log n)	**30**	**30**	**40**	**100**